

Open-Source Simulators for Multi-Domain Ocean Robotics: An Annotated Literature Review

Brian Bingham*

September 9, 2026

Abstract

This is a working literature review of open-source simulation frameworks usable for ocean robotics in the year 2026. The scope is projects that can represent the marine domain together with adjacent domains (underwater, water surface, air, land), and that satisfy our working definition of a healthy software project: open source, well documented, and supported by a sizable, active community. Projects whose domain-specific layer is itself open source are included even when they sit on closed-source engines (Unreal, Unity, NVIDIA Isaac Sim). Each entry has a short note that says what the project is and why we keep it on the reading list. The aim is a document short enough to keep current and verbose enough that a future contributor can recover the reasoning without reading the underlying papers again.

1 Overview articles

Before the project-by-project notes, it is worth knowing who else has surveyed this ground and on what terms. The entries below are review and comparison papers rather than simulator releases, listed newest first. Each is summarized the same way: whose vantage point the assessment is written from, and what the paper actually measures. The short version is that the review literature is dominated by the underwater perception problem, that surface and multi-domain operation is covered thinly or not at all, and that almost all of it compares declared features rather than measured behavior.

Simulation Platforms for Underwater Robotic Applications (Shaw et al. 2026), IIT-Mandi, *Journal of Field Robotics*, is the longest of these—33 journal pages—and the most systematic.

- Special focus: "hydrodynamic modeling, visual rendering, and the simulation of realistic underwater phenomena like turbidity, wave interactions, and marine habitat dynamics." This list combines disparate items. Hydrodynamics are a given for any sim including vessels at or below the surface, rendering is important for any simulation - and then the underwater phenomena are oddly specific.

The perspective is a software-architecture teardown: the first half decomposes a generic underwater simulator into six modules (environmental design, vehicle modeling and dynamics, sensors and actuators, internal communication, external/acoustic communication, and the surrounding framework), defines its terms down to what separates a library from an API from a framework, and writes out the six-DOF Fossen equations that the dynamics modules implement. The second half then organizes the field *genealogically* rather than by paradigm—UWSim and its descendants

*briansbingham@gmail.com. Additional contributors and reviewers are listed in Section 7.

(**freefloating-gazebo**, USVSim), UUVSim and its descendants (DAVE), UW-MORSE, Unity-based (URSim), Unreal-based (HoloOcean, UNav-Sim), and a residual group (MARS, Stonefish, ROCK-Gazebo, WAVE, OysterSim, ChatSim)—describing, for each extension, only the modules it changed.

Three things in it are not available anywhere else in this literature. Its Table 3 records, per simulator, the developing institution, release date, date of last commit, and a three-valued maintenance status (active / partial / inactive) read off GitHub, alongside minimum CPU, GPU, VRAM, RAM, and supported OS—the only published attempt to say what these tools cost to *run* and whether they are still alive, which is close to our own community-health test in Section 3. Its Table 6 lists the real vehicle each simulator has actually been validated against. And Section 6 proposes a preliminary benchmarking framework (Table 5) with concrete tasks and quantitative metrics—trajectory error, task success rate, robustness to disturbance, computational efficiency—rather than merely calling for one, as everyone else does. Its stated future directions are worth noting too: it argues simulators should model biological and human-made disturbance (fish, biofouling, nets, ropes, ship noise, debris), not just waves and currents.

Marine Robotic Simulators (Sim et al. 2026), by Sim, Kweon and Joe at Kyungpook National University, is written from the synthetic-data vantage point: their organizing claim is that simulator requirements are now set by deep reinforcement learning and visual SLAM rather than by vehicle dynamics, and that the field has reorganized itself around rendering and sensor realism as a result. The method is explicit—a keyword search over IEEE Xplore, Scopus, and Google Scholar, exclusion of anything not marine-specific, then one consistent rubric (physics modeling, sensor simulation, rendering capability, system integration) applied across every entry in four comparison tables, one of them a sensor-by-sensor matrix. They sort the field into five generational paradigms: legacy (UWSim, UUV Simulator), physics-focused (Stonefish, DAVE), game-engine (HoloOcean, UNav-Sim, MARUS, FURo-Sim), GPU-native/Omniverse (OceanSim), and acoustics-first tools such as ACSim. The closing discussion names the realism-versus-throughput trade-off and the absence of a marine equivalent to KITTI or Gym as the field’s two structural problems.

Underwater Robotic Simulators Review (Aldhaheri et al. 2025), from UCL and the Technology Innovation Institute, is written as a practitioner’s selection guide and is the only survey here that states its inclusion gates as plainly as we do in Section 3: native ROS support, evidence of active maintenance, and an open-source license. Those gates cut the field to five systems—Stonefish, DAVE, HoloOcean, MARUS, UNav-Sim—compared on release year, OS support, ROS integration, physics engine, manipulator support, multi-vehicle support, acoustic communications, and perception realism. Its distinctive contribution is bibliometric rather than technical: the authors hand-classify over 240 downstream studies into perception, planning, control, and mentions-only, and use the result as a proxy for community traction. Read that figure with care—it is a raw citation count as of Q1 2025, so it rewards age, and Stonefish’s lead over UNav-Sim is partly a four-year head start. Its conclusions land close to ours: that simulators built on ROS 1 risk obsolescence without a deliberate migration, and that the absence of a standardized open benchmark is what blocks reproducibility. This is the survey referenced in Section 6 as a companion reference.

URoBench (Huang et al. 2024), from Heriot-Watt and Edinburgh, is the outlier: it is the only paper here that runs the simulators and reports numbers. The authors first shortlist qualitatively—physical modeling, sensor modeling, API usability, documentation, multi-agent support—and then build an open-source Gym wrapper with per-simulator connectors that puts HoloOcean, DAVE, and Stonefish behind one reinforcement-learning interface, containerized in Docker, so that identical tasks (a “cruise” waypoint task and a “follow” trajectory task) can be run on each. Four metrics are logged continuously: real-time factor, CPU, GPU, and memory, in both GUI and headless modes, over ten runs of 1000 steps at a 10 ms timestep on a single desktop.

The results are worth knowing before designing anything around throughput. HoloOcean turns in the highest real-time factor and the lowest CPU load; DAVE, on Gazebo, is the most CPU- and memory-hungry, with `gzclient` and `gzserver` accounting for the GUI-mode gap; Stonefish has the smallest memory footprint and is the only one that meaningfully exercises the GPU. None of the three reaches real time—every reported RTF is below 1, even headless—which should temper any assumption that these tools are ready to be run faster than real time or many-at-once. Two caveats the authors state plainly and that anyone citing this should carry: the figures measure the simulator *plus the harness*, and the ROS pause/resume service round-trip that the harness needs is itself a significant cost, so HoloOcean’s advantage partly reflects its native Python interface rather than a faster engine; and HoloOcean’s multiple-ticks-per-step feature was disabled to keep timing uniform. It is the closest published work to the vectorized-training use case in Section 2 and a useful corrective to feature-matrix surveys, which cannot say what a simulator costs to run.

Perspective on the Marine Simulator for Autonomous Vessel Development (Sawada et al. 2024), from Japan’s National Maritime Research Institute and Osaka University, is the surface-vessel counterweight to the underwater literature, and it reviews a different axis entirely: not simulator software, but the ship maneuvering models that sit inside one. The perspective is naval architecture and regulatory approval for maritime autonomous surface ships. It covers response models (Nomoto’s KT model and its extensions), autoregressive and neural-network models, and fluid-dynamics models (whole-ship and modular MMG), then the four routes to hydrodynamic derivatives: type-ship values, empirical databases, captive model tests, and system identification from maneuvering trials. The assessment criterion is fitness for the approval purpose—measured against class-society guidelines, the DNV maritime simulator standard, and IEC 62065—traded against the cost of building the model. Worth reading because it treats fidelity as a certification question rather than a rendering question, which is the framing our fourth use case (mission-plan validation) will eventually need.

A Review of Physics Simulators for Robotic Applications (Collins et al. 2021), from CSIRO and QUT, is not a marine review but a whole-of-robotics index, organized by research sub-community—mobile ground, manipulation, medical, marine, aerial, soft, and learning—with a feature table for each. Marine robotics is one section, split into underwater (UWSim, UUV Simulator, Stonefish, ROCK-Gazebo, `freefloating-gazebo`, URSim) and surface (USVSim), with the explicit observation that surface-vehicle simulation is rare because waves, wind, and current are hard to model. We keep it because it is the reference that shows how thin the marine column is next to the aerial and manipulation ones—the same asymmetry that motivates this document.

A Survey of AUV and Robot Simulators for Multi-Vehicle Operations (Cook et al. 2014), from Memorial University of Newfoundland, was the first survey to treat multi-vehicle operation as the selection driver rather than an afterthought. Criteria are stated up front: physical fidelity sufficient for manipulators and end effectors, a programmatic interface via scripting or middleware, optical and/or acoustic sensor models, adequate documentation, and prior use in published research. It surveys the general-purpose field and then goes deep on UWSim, MORSE, and Gazebo, with a separate section on re-purposing game engines that reads as a direct antecedent of HoloOcean and MARUS a decade later.

Two of its judgments are worth carrying forward. First, it explicitly *deprioritizes* visual fidelity, on the argument that most apparent rendering differences come from the textures and models a project happens to ship rather than from the simulator, so that the rendering capability of most simulators is the same once user-replaceable assets are set aside. That inverts the axis on which the 2025 and 2026 surveys principally rank the field, and it is a claim worth re-examining now that rendering is entangled with sensor synthesis rather than with looks. Second, its discussion names abandonment risk and packaging lag—academic open-source projects going quiet as researchers

move on, binary releases falling behind the current ROS distribution—as the practical failure mode of open-source simulators. That is the concern behind our own community-size and maintenance criterion in Section 3, stated twelve years earlier. It builds on two earlier surveys, Matsebe et al. (2008) below and a 2007 survey of unmanned-vehicle simulators by Craighead et al., and concludes that underwater simulation “remains a niche problem.”

A Review of Virtual Simulators for Autonomous Underwater Vehicles (Matsebe et al. 2008), from CSIR South Africa and Tshwane University of Technology, is the historical baseline, written when the problem was simply that no one could tell which tools existed. The authors are candid that the selection reflects their own preference. None of the seven systems it covers—MVS, CADCON, IGW, NEPTUNE, DVECS, SUBSIM, DeepWorks—survives today, and most were internal academic projects never released, which is why Cook et al. (2014) later set it aside.

Its taxonomy, though, has aged better than its contents, because it classifies by *how the simulator couples to real time and real hardware* rather than by how it looks: offline (MATLAB/Simulink-style, no environment or external sensors, and no guarantee that a simulated second matches a real one), online (real-time, consistency between simulation and reality guaranteed), hardware-in-the-loop, and hybrid, in which a real vehicle in a test tank flies against virtual sensors and virtual obstacles. That is very nearly the question our second and fourth use cases ask, and it is the axis the modern surveys have quietly dropped in favor of rendering and sensor realism.

What the review literature does not cover

Three gaps are worth naming, because they are the reasons this document exists alongside the surveys above.

- **Multi-domain operation is absent.** Every marine survey here is underwater-first. Surface vehicles get a short subsection in Collins et al. (2021), a single entry (USVSim) inside the UWSim family tree in Shaw et al. (2026), and the whole of Sawada et al. (2024)—but no review assesses a simulator’s ability to carry underwater, surface, aerial, and terrestrial assets in one world, and none treats the air-water boundary as a modeling problem. VRX is not evaluated in any of them; neither is any aerial or terrestrial capability of a marine stack.
- **Almost nothing is measured.** Huang et al. (2024) is the only paper here that runs the simulators and reports numbers, and it covers three of them on one desktop. Shaw et al. (2026) proposes a benchmarking framework and tabulates hardware requirements and maintenance status, which is the next best thing, but does not execute it. Everything else compares declared features, which says nothing about determinism, throughput, or headless stability. Note also that the one measured result—real-time factor below 1 for all three simulators tested, even headless—sits awkwardly with how confidently the rest of the literature discusses large-scale RL training.
- **CI/CD and long-horizon mission validation are not assessment criteria anywhere.** No survey asks whether a simulator can gate a merge or run a thirty-hour mission faster than real time. The closest anyone comes is the offline/online/HIL/hybrid taxonomy of Matsebe et al. (2008), from 2008. Those are use cases two and four in Section 2.

The PDFs of the papers above are kept as a local reading cache in `lit/` and are deliberately not committed to the repository. `lit/MANIFEST.md` is tracked and lists, for each entry, either the command that fetches it or the DOI to request through a library.

2 Scope and motivation

We care about simulators that have to serve four use cases. Each one puts different pressure on the underlying tool.

1. **Manual, in-the-loop development and operator training.** The simulator is a visual and dynamic sandbox in which a human iterates on algorithms, missions, or operator procedures. What matters here is rendering, asset availability, and ergonomics. Raw throughput is secondary.
2. **Continuous-integration regression testing for software safety.** The simulator has to run headless, has to be deterministic, has to be fast enough to gate merges, and has to stay stable across OS upgrades. What matters here is reproducibility and packaging. Visual polish does not.
3. **Vectorized training for machine-learning research.** The simulator is treated as a parallelizable environment factory. What matters here is steps per wall-second across many concurrent worlds, GPU acceleration, deterministic resets, and a Python-native interface.
4. **Mission-plan validation.** The simulator runs a full mission end to end to catch faults in the *plan*—grounding, collision, energy- or time-budget blowouts, contingency-logic errors, missed communication windows—before deployment. What matters here is faster-than-real-time, long-horizon, deterministic execution, since ocean missions run for tens of hours to weeks. This is the use case where the marine domain diverges most sharply from its short-mission aerial and ground neighbors.

Our domain of interest is *multi-domain ocean robotics*: a single simulated world that may contain assets operating underwater, on the water surface, in the air, and on land, interacting through a shared environment (for example, a USV launching a UAV that overflies a shoreline sensor while an AUV operates below the surface).

3 Selection criteria

We use three working criteria for inclusion. A project is in scope if it is (i) **open source** with a usable license; (ii) has **documentation good enough that a new user can stand it up without reading the source** (a soft criterion; we note when only the source serves as documentation); and (iii) shows **evidence of a sizable, active community**, taken here as some combination of GitHub stars, recent commit cadence, a maintained forum or chat, and outside citations.

Projects built on proprietary engines (NVIDIA Isaac Sim, Unreal, Unity) are included when the marine layer is itself open source and meets the three criteria above. We **exclude** projects that are one-off academic prototypes without a sustained user base. A small number of borderline cases are noted in Section 6 so the document can be revisited as those communities mature.

4 The landscape

4.1 General-purpose substrate: Gazebo

Modern Gazebo (Koenig and Howard 2004) is the substrate beneath most of the open-source marine stacks we cover. The Open-Robotics-led modern series (Garden, Harmonic, Jetty) replaced the “Ignition” naming after the 2022 trademark issue. Gazebo Classic reached end-of-life in January 2025. The project is a case of large community plus thin documentation: a clean API and a sprawling ecosystem, but learning paths still tend to route through source reading and community channels.

4.2 Gazebo-based marine stacks

The largest cluster of marine simulators builds on the Gazebo family.

UUV Simulator (Manhães et al. 2016) pioneered Gazebo Classic underwater dynamics and is still widely cited, even though the upstream repository was archived in 2023. We keep it here for historical context: it is the genealogical root of most ROS/Gazebo underwater work.

DAVE (Zhang et al. 2022) extends UUV Simulator toward standardized scenes, manipulation, and oceanographic environments. The project is mid-port from Gazebo Classic to modern Gazebo and is led by a group that includes the Naval Postgraduate School, WHOI, and Open Robotics.

VRX (Virtual RobotX) (Bingham et al. 2021) is the most actively maintained marine Gazebo stack we know of. It was originally built for the RobotX Maritime Challenge; modern releases (v3.x, on Gazebo Harmonic and ROS 2 Jazzy) run surface, aerial, and terrestrial assets together in one world. Of the projects in this review, VRX comes closest to delivering the multi-domain story end to end on an open stack.

LRAUV simulation (Player et al. 2023) from MBARI is a production-targeted digital twin used to validate long-duration science missions before deployment. The standalone `osrf/lrauv` fork has been upstreamed into Gazebo and was archived in 2025; ongoing development continues in the MBARI organization.

4.3 Game-engine-based simulators

HoloOcean (Potokar, Ashford, et al. 2022; Potokar, Lay, et al. 2024) is BYU’s Unreal-Engine-based underwater simulator and the most cited non-Gazebo option in marine perception research. Its 2024 *Journal of Oceanic Engineering* release broadens the sensor suite (imaging sonar, DVL, side-scan, multibeam). A 2.0 preview targets Unreal 5.3 with a ROS 2 bridge.

OceanSim (Song et al. 2025) from the Field Robotics Group at the University of Michigan ports the underwater perception problem onto NVIDIA Isaac Sim, inheriting GPU ray tracing and the Omniverse asset pipeline. It is the most credible candidate in this list for the third use case in Section 2 (vectorized ML training). At the time of writing it is underwater-only, but the underlying engine supports terrestrial and aerial assets, so the gap is one of marine-domain content rather than engine capability.

4.4 Standalone hydrodynamics + rendering

Stonefish (Cieślak 2019; Grimaldi, Cieślak, et al. 2025) is a self-contained C++/OpenGL simulator from the University of Girona, with strong manipulator support and ROS 1/ROS 2 wrappers. Its 2025 ICRA paper adds ML-oriented sensors (event camera, thermal, optical flow, visible-light communication) and frames the project explicitly as a substrate for machine-learning research in marine robotics. The license is GPL-3.0, which is more restrictive than the Apache-licensed Gazebo stack and matters for downstream redistribution.

4.5 Hydrodynamics specialist

WEC-Sim (Ruehl et al. 2017) (NREL + Sandia, funded by the US DOE Water Power Technologies Office) is the canonical open-source wave-energy-converter modeling toolbox. It is a MATLAB/Simulink toolchain rather than a robotics simulator, but it is the reference for high-fidelity floating-body hydrodynamics and is worth keeping on the reading list as a verification target for the surface dynamics of robotics simulators.

5 Cross-cutting observations

- **Only VRX currently runs underwater, surface, aerial, and terrestrial dynamics in a single maintained open stack.** Its underwater coverage is shallow next to the perception fidelity of HoloOcean or OceanSim, but the multi-domain integration story is uncommon.
- **Vectorized ML training is best supported on the Isaac-Sim branch (OceanSim) and, to a lesser extent, HoloOcean.** Gazebo’s throughput model does not yet match the thousands-of-parallel-worlds idiom that dominates modern RL.
- **CI/CD adoption favors Gazebo today.** Deterministic headless runs through `gz sim` are mature, and the same binaries are used in development and CI. Unreal- and Unity-based simulators are harder to package for unattended runs.
- **Documentation quality varies a lot.** WEC-Sim, HoloOcean, and VRX have approachable user-facing documentation. Gazebo, DAVE, and Stonefish rely more on source reading and community channels.
- **License diversity matters.** Most of the Gazebo ecosystem and OceanSim are Apache-2.0. HoloOcean is MIT but inherits the Unreal EULA at runtime. Stonefish is GPL-3.0. Downstream redistribution plans need to account for this.

6 Excluded and adjacent

The projects below come up in marine-simulator discussions but do not currently meet the community-size criterion, so they are noted here rather than reviewed.

- **MARUS** (Lončar et al. 2022) — a Unity-based maritime simulator from the University of Zagreb. Promising technical direction but limited adoption outside its originating lab.
- **Project Aquaticus** (Novitzky et al. 2024) — a maritime capture-the-flag testbed originally from MIT and now at USMA West Point, built on MOOS-IvP rather than a 3-D physics simulator. Cite for human-robot teaming context, not as a simulator product.
- **Cosys-AirSim / ASVSim** (Jansen et al. 2023; Middelaar et al. 2025) — the University of Antwerp’s active fork of Microsoft AirSim (archived 2022). Mostly aerial, with a 2025 ASV spinoff. Worth monitoring; revisit if the marine layer matures.

Umbrella references that cover many of the same systems—in particular Aldhaheri et al. (2025) and Sim et al. (2026)—are reviewed in Section 1. We treat them as companion documents rather than replacements for project-by-project assessment.

7 Contributors and contact

This is a living working review. Contributors are listed below with the area they were responsible for. Please contact the relevant person before making substantive edits in their section so the annotations stay coherent.

Contributor	Contact	Area
Brian Bingham	briansbingham@gmail.com	Overall editor; Gazebo and VRX sections
(open)	(open)	Underwater perception (HoloOcean, OceanSim, Stonefish)
(open)	(open)	Hydrodynamics and surface dynamics (WEC-Sim, LRAUV)
(open)	(open)	ML training infrastructure and vectorized environments

Annotated references

Aldhaheri, Sara, Yang Hu, Yongchang Xie, Peng Wu, Dimitrios Kanoulas, and Yuanchang Liu (2025). “Underwater Robotic Simulators Review for Autonomous System Development”. In: *Proc. OCEANS 2025 Brest*, pp. 1–10. DOI: [10.1109/OCEANS58557.2025.11104581](https://doi.org/10.1109/OCEANS58557.2025.11104581). arXiv: [2504.06245](https://arxiv.org/abs/2504.06245). URL: <https://frl-ucl.github.io/projects/underwater-robotic-simulators.html> (visited on 09/09/2026).

Notes. *University College London and the Technology Innovation Institute, Abu Dhabi. Written as a practitioner’s selection guide, and the only survey here that states its inclusion gates as plainly as we do in Section 3: native ROS support, evidence of active maintenance, and an open-source license. Those gates cut the field to five systems—Stonefish, DAVE, HoloOcean, MARUS, UNav-Sim—compared on release year, OS support, ROS integration, physics engine, manipulator support, multi-vehicle support, acoustic communications, and perception realism. Its distinctive contribution is bibliometric: a hand-classified sweep of 240+ downstream studies binned into perception, planning, control, and mentions-only, used as a proxy for community traction. Stonefish comes out ahead there, but the figure is a raw Q1 2025 citation count and so rewards age; its lead over UNav-Sim is partly a four-year head start. Conclusions land close to ours: ROS 1-era simulators risk obsolescence without deliberate migration, and the missing piece is a standardized open benchmark. Note that an earlier draft of our bibliography carried this work under a wrong author name; the entry has been corrected.*

Bingham, Brian, Carlos Agüero, Michael McCarrin, Joseph Klamo, Joshua Malia, Kevin Allen, Tyler Lum, Marshall Rawson, and Rumman Waqar (2021). “Toward Maritime Robotic Simulation in Gazebo”. In: *Proc. OCEANS 2021 San Diego–Porto*. DOI: [10.23919/OCEANS44145.2021.9705971](https://doi.org/10.23919/OCEANS44145.2021.9705971). URL: <https://github.com/osrf/vrx> (visited on 05/21/2026).

Notes. *Reference open-source USV simulator and the backbone of the RobotX Maritime Challenge. Modernization (v3.x on Gazebo Harmonic + ROS 2 Jazzy) is led by Honu Robotics; sponsored by RoboNation. Apache-2.0. The only project we survey that integrates surface, aerial, and terrestrial dynamics in one open, maintained world — which is why it is central to a multi-domain ocean review.*

Cieślak, Patryk (2019). “Stonefish: An Advanced Open-Source Simulation Tool Designed for Marine Robotics, With a ROS Interface”. In: *Proc. OCEANS 2019 Marseille*. DOI: [10.1109/OCEANSE.2019.8867434](https://doi.org/10.1109/OCEANSE.2019.8867434). URL: <https://github.com/patrykcieślak/stonefish> (visited on 05/21/2026).

Notes. *Self-contained C++/OpenGL simulator from the Underwater Vision and Robotics Lab at the University of Girona (CIRS). Independent of Gazebo, with its own renderer and Bullet physics. GPL-3.0; ROS 1/ROS 2 wrappers via `stonefish_ros`. Included because Stonefish is the most active non-game-engine alternative for high-fidelity underwater simulation.*

Collins, Jack, Shelvin Chand, Anthony Vanderkop, and David Howard (2021). “A Review of Physics Simulators for Robotic Applications”. In: *IEEE Access* 9, pp. 51416–51431. DOI: [10.1109/ACCESS.2021.3068769](https://doi.org/10.1109/ACCESS.2021.3068769). (Visited on 09/09/2026).

Notes. CSIRO and QUT. Not a marine review—a whole-of-robotics index organized by research sub-community (mobile ground, manipulation, medical, marine, aerial, soft, learning), each with its own feature table. Marine robotics is Section V and Table 5, split into underwater (UWSim, UUV Simulator, Stonefish, ROCK-Gazebo, freefloating-gazebo, URSim) and surface (USVSim), with the explicit observation that surface-vehicle simulation is rare because waves, wind, and current are hard. Kept here because it is the reference that lets us see how thin the marine column is next to the aerial and manipulation ones—the same asymmetry that motivates this review. Open access, CC BY.

Cook, Daniel, Andrew Vardy, and Ron Lewis (2014). “A Survey of AUV and Robot Simulators for Multi-Vehicle Operations”. In: *Proc. IEEE/OES Autonomous Underwater Vehicles (AUV)*, pp. 1–8. DOI: [10.1109/AUV.2014.7054411](https://doi.org/10.1109/AUV.2014.7054411). (Visited on 09/09/2026).

Notes. Memorial University of Newfoundland. The first survey to treat multi-vehicle operation as the selection driver rather than an afterthought. Inclusion criteria are stated up front: physical fidelity sufficient for manipulators and end effectors, a programmatic interface via scripting or middleware, optical and/or acoustic sensor models, adequate documentation, and prior use in published research. Surveys the general-purpose field, then goes deep on UWSim, MORSE, and Gazebo, with a separate section on re-purposing game engines (Unity, Leadwerks, Unreal) that reads as a direct antecedent of HoloOcean and MARUS. Two judgments worth carrying forward: it deliberately deprioritizes visual fidelity, arguing that apparent rendering differences come mostly from shipped assets rather than from the simulator—the inverse of how the 2025–2026 surveys rank the field—and its discussion names abandonment risk and binary-packaging lag as the practical failure mode of academic open-source simulators, which is our community-health criterion stated twelve years early. Builds on Craighead et al. (2007) and Matsebe, Kumile, and Tlale (2008); concludes underwater simulation “remains a niche problem.” Paywalled.

Grimaldi, Michele, Patryk Cieślak, et al. (2025). “Stonefish: Supporting Machine Learning Research in Marine Robotics”. In: *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*. arXiv: [2502.11887](https://arxiv.org/abs/2502.11887). URL: <https://github.com/patrykcieślak/stonefish> (visited on 05/21/2026).

Notes. Adds ML-oriented sensors (event camera, thermal, optical flow, visible-light communication) and frames Stonefish as an ML research substrate. Cite this rather than the 2019 OCEANS paper when discussing ML use cases.

Huang, Zebin, Markus Buchholz, Michele Grimaldi, Hao Yu, Ignacio Carluchó, and Yvan R. Petillot (2024). “URoBench: Comparative Analyses of Underwater Robotics Simulators from Reinforcement Learning Perspective”. In: *Proc. OCEANS 2024 Singapore*, pp. 1–8. DOI: [10.1109/OCEANS51537.2024.10682284](https://doi.org/10.1109/OCEANS51537.2024.10682284). (Visited on 09/09/2026).

Notes. Heriot-Watt University and the University of Edinburgh. The only entry in this section that measures rather than tabulates. Qualitative shortlist first (physical modeling, sensor modeling, API usability, documentation, multi-agent support), then an open-source Gym wrapper plus per-simulator connectors, Docker-contained, that puts HoloOcean, DAVE, and Stonefish behind one RL interface; identical cruise and trajectory-following tasks are run on each while RTF, CPU, GPU, and memory are logged in GUI and headless modes (10 runs, 1000 steps, 10 ms timestep, i5-10400/RTX 3060). Headline results: HoloOcean fastest and lightest on CPU, DAVE heaviest on CPU and memory, Stonefish smallest memory footprint and the only real GPU user—but every measured RTF is below unity, even headless. Two caveats the authors state and that should travel with any citation: the numbers are simulator-plus-harness, and the ROS pause/resume round-trip the harness needs is itself costly, so HoloOcean partly benefits from a native Python interface rather than a faster engine; and its multiple-ticks-per-step feature was disabled for timing uniformity. Code at <https://github.com/oceansystemslab/URoBench>. Paywalled; no preprint located.

Jansen, Wouter, Erik Verreycken, Anthony Schenck, Jean-Edouard Blanqué, Henri Verhulst, and Jan Steckel (2023). *Cosys-AirSim: A Real-Time Simulation Framework Expanded for Complex*

- Industrial Applications*. arXiv: 2303.13381. URL: <https://github.com/Cosys-Lab/Cosys-AirSim> (visited on 05/21/2026).
- Notes.** University of Antwerp’s actively maintained fork of Microsoft AirSim (archived 2022). MIT-licensed; predominantly aerial. Noted because AirSim derivatives are the most common path to marine simulation on Unreal outside of HoloOcean.
- Koenig, Nathan and Andrew Howard (2004). “Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator”. In: *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Sendai, Japan, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727. URL: <https://github.com/gazebosim/gz-sim> (visited on 05/21/2026).
- Notes.** Founding paper for Gazebo, now maintained by Open Robotics (post-Ignition rename). Modern Gazebo (Garden, Harmonic, Jetty) is the substrate beneath VRX, DAVE, and LRAUV. Gazebo Classic reached end-of-life Jan 2025. Apache-2.0; very active community via gazebosim.org forum, Discord, monthly community meetings. Included as the de-facto open substrate for ROS-based marine simulation.
- Lončar, Igor, Juraj Obradović, Natko Krčmar, Ivan Sitar, Luka Kušnjier, Nikola Mišković, and Zoran Vukić (2022). “MARUS – A Marine Robotics Simulator”. In: *Proc. OCEANS 2022 Hampton Roads*. DOI: 10.1109/OCEANS47191.2022.9977075. URL: <https://github.com/MARUSimulator> (visited on 05/21/2026).
- Notes.** Unity-based maritime simulator from LABUST, University of Zagreb FER. Apache-2.0; gRPC bridge to ROS 1/ROS 2. Technically interesting but with limited adoption outside the originating lab. Noted here so we can revisit if the community grows.
- Manhães, Musa Morena Marcusso, Sebastian A. Scherer, Martin Voss, Luiz Ricardo Douat, and Thomas Rauschenbach (2016). “UUV Simulator: A Gazebo-based package for underwater intervention and multi-robot simulation”. In: *Proc. OCEANS 2016 MTS/IEEE Monterey*. Monterey, CA, USA. DOI: 10.1109/OCEANS.2016.7761080. URL: https://github.com/uuvsimulator/uuv_simulator (visited on 05/21/2026).
- Notes.** The genealogical root of the open-source Gazebo+ROS underwater stack. Repository archived Feb 2023 and the project is unmaintained, but it is the upstream of DAVE and is still the most commonly cited paper for AUV simulation in Gazebo. Apache-2.0. Kept for historical and didactic value, not as a recommended current tool.
- Matsebe, Oduetse, Christiaan M. Kumile, and Nkgatho S. Tlale (2008). “A Review of Virtual Simulators for Autonomous Underwater Vehicles (AUVs)”. In: *IFAC Proceedings Volumes (2nd IFAC Workshop on Navigation, Guidance and Control of Underwater Vehicles)*. Vol. 41. 1, pp. 31–37. DOI: 10.3182/20080408-3-IE-4914.00007. (Visited on 09/09/2026).
- Notes.** CSIR South Africa and Tshwane University of Technology. The historical baseline, written when the problem was simply that no one could tell which tools existed; the authors are candid that the selection reflects their own preference. Covers MVS, CADCON, IGW, NEPTUNE, DVECS, SUBSIM, and DeepWorks, none of which survives—most were unreleased internal academic projects, which is why Cook, Vardy, and Lewis (2014) later set this survey aside. The taxonomy has aged far better than the contents: it classifies by how a simulator couples to real time and real hardware—offline (no environment, no external sensors, no guarantee a simulated second matches a real one), online (real-time, consistent with reality), hardware-in-the-loop, and hybrid (a real vehicle in a test tank flying against virtual sensors and obstacles). That axis is close to what our CI and mission-validation use cases actually ask, and the modern surveys have largely dropped it in favor of rendering and sensor realism. Paywalled.
- Middelaar, Robin van et al. (2025). *ASVSim: A Simulation Framework for Autonomous Surface Vehicles Built on Cosys-AirSim*. arXiv: 2506.22174. URL: <https://arxiv.org/abs/2506.22174> (visited on 05/21/2026).
- Notes.** Surface-vehicle spinoff of Cosys-AirSim. Too new (2025) to claim community traction; included so we can revisit the Unreal+ASV story in a future pass.
- Novitzky, Michael, Charles Pippin, Thomas R. Collins, Paul Fitzpatrick, Alexander Holness, and Henrik Schmidt (2024). *Evaluating Collaborative Autonomy in Opposed Environments Using*

Maritime Capture-the-Flag Competitions. arXiv: 2404.17038. URL: <https://oceanai.mit.edu/aquaticus/> (visited on 05/21/2026).

Notes. *Project Aquaticus is a maritime capture-the-flag testbed originally developed at MIT and migrated to USMA West Point in 2019. Built on MOOS-IvP rather than a 3-D physics simulator; cite for human-robot teaming context, not as a comparable simulator product.*

Player, Timothy R., Arjo Chakravarty, Mabel M. Zhang, Ben Yair Raanan, Brian Kieft, Yanwu Zhang, and Brett Hobson (2023). “From Concept to Field Tests: Accelerated Development of Multi-AUV Missions Using a High-Fidelity Faster-than-Real-Time Simulator”. In: *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*. London, UK. URL: https://github.com/mbari-org/gz_lrauv (visited on 05/21/2026).

Notes. *Production digital twin of MBARI’s Tethys-class long-range AUV, used to validate multi-day science missions before deployment. Demonstrates Gazebo (modern) for long-duration AUV ops. The standalone `osrf/lrauv` fork was archived in Mar 2025 after upstreaming; ongoing work continues in the MBARI org. Apache-2.0. Included as an exemplar of production use of an open simulator.*

Potokar, Easton, Spencer Ashford, Michael Kaess, and Joshua G. Mangelson (2022). “HoloOcean: An Underwater Robotics Simulator”. In: *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*. Philadelphia, PA, USA. DOI: 10.1109/ICRA46639.2022.9812388. URL: <https://byu-holocean.github.io/holocean-docs/> (visited on 05/21/2026).

Notes. *Initial release of BYU FROSTLab’s Unreal-Engine underwater simulator, built on top of Microsoft Holodeck. Python-first API. MIT license on the client side (Unreal EULA applies to the engine). Kept because HoloOcean is the most widely cited non-Gazebo underwater simulator in marine perception research.*

Potokar, Easton, Kalin Lay, Kalebh Norman, Devin Benham, Spencer Ashford, Robert Peirson, and Joshua G. Mangelson (2024). “HoloOcean: A Full-Featured Marine Robotics Simulator for Perception and Autonomy”. In: *IEEE Journal of Oceanic Engineering*. DOI: 10.1109/JOE.2024.3403060. URL: <https://froslab.byu.edu/holocean-underwater-simulator> (visited on 05/21/2026).

Notes. *Expanded sensor suite (imaging sonar, side-scan, multibeam, DVL) and improved hydrodynamics. The reference paper to cite for current HoloOcean capabilities. A 2.0 preview targeting Unreal 5.3 and ROS 2 appeared in late 2025 (arXiv:2510.06160) but is not yet a stable release.*

Ruehl, Kelley, Carlos Michelen, Bret Bosma, and Yi-Hsiang Yu (2017). “WEC-Sim: An Open-Source Tool for Simulating Wave Energy Converters”. In: *Proc. 12th European Wave and Tidal Energy Conf. (EWTEC)*. Cork, Ireland. URL: <https://wec-sim.github.io/WEC-Sim/> (visited on 05/21/2026).

Notes. *NREL + Sandia, funded by US DOE Water Power Technologies Office. MATLAB/Simulink/Simscape Multibody toolchain for floating-body hydrodynamics; not a robotics simulator. Apache-2.0; v7.0 released Aug 2025; runs an annual training course and active forum. Included as the domain standard for surface hydrodynamics — useful as a verification reference, not as a primary robotics simulator.*

Sawada, Ryouhei, Yoshiki Miyauchi, Suisei Wada, Takuya Tanigushi, Satoru Hamada, Hiroaki Koike, Kouki Wakita, and Atsuo Maki (2024). “Perspective on the Marine Simulator for Autonomous Vessel Development”. In: *Journal of Marine Science and Technology*. arXiv: 2407.19673. URL: <https://arxiv.org/abs/2407.19673> (visited on 09/09/2026).

Notes. *National Maritime Research Institute (Japan) and Osaka University. The surface-vessel counterweight to the underwater surveys, and it reviews a different axis entirely: not simulator software, but the ship maneuvering models that sit inside one. Covers response models (Nomoto KT and extensions), autoregressive and neural-network models, and fluid-dynamics models (whole-ship and modular MMG), then the four routes to hydrodynamic derivatives—type-ship values, empirical databases, captive model tests, and system identification from maneuvering trials. Assessment criterion is fitness for the approval purpose against class-society guidelines, the DNV maritime simulator standard, and IEC 62065, traded*

off against the cost of building the model. Worth reading precisely because it treats fidelity as a certification question rather than a rendering question.

Shaw, Subham Kumar, Prasanna Muppidwar, and Jagadeesh Kadiyam (2026). “Simulation Platforms for Underwater Robotic Applications: Architectures, Capabilities, and Research Directions”. In: *Journal of Field Robotics* 43.6, pp. 4116–4148. DOI: [10.1002/rob.70234](https://doi.org/10.1002/rob.70234). (Visited on 09/09/2026).

Notes. The broadest and most systematic of the recent underwater surveys at 33 journal pages. Centre for Artificial Intelligence and Robotics, IIT Mandi, with G. H. Raison College of Engineering. First half is a module-by-module teardown of a generic underwater simulator (environment, vehicle dynamics via the six-DOF Fossen equations, sensors and actuators, internal and acoustic communication, framework); second half organizes the field by genealogy rather than paradigm—UWSim and its descendants (**freefloating-gazebo**, USVSim), UUVSim and DAVE, UW-MORSE, Unity-based (URSim), Unreal-based (HoloOcean, UNav-Sim), plus MARS, Stonefish, ROCK-Gazebo, WAVE, Oyster-Sim, ChatSim—documenting for each extension only the modules it changed. Three things are unique to it: Table 3 pairs minimum CPU/GPU/VRAM/RAM and OS support with last-commit date and a three-valued maintenance status read off GitHub; Table 6 names the real vehicle each simulator has been validated against; and Section 6 proposes an actual benchmarking framework (Table 5) with quantitative metrics—trajectory error, task success rate, robustness to disturbance, computational efficiency—rather than only calling for one. Also argues, unusually, that simulators should model biological and anthropogenic disturbance (fish, biofouling, nets, ship noise, debris). Paywalled.

Sim, Hyeonmin, Heekyu Kweon, and Hangil Joe (2026). “Marine Robotic Simulators: A Comprehensive Review of Architectures, Sensor Modeling, and Sim-to-Real Challenges”. In: *InUS Transactions* 2.1, pp. 17–32. DOI: [10.23380/inustrans.2026.2.1.2](https://doi.org/10.23380/inustrans.2026.2.1.2). URL: <https://acoms.accesson.kr/inustrans> (visited on 09/09/2026).

Notes. Kyungpook National University. Published March 2026 and, at the time of writing, the most current umbrella reference for the underwater side. Explicitly methodological: keyword search over IEEE Xplore, Scopus, and Google Scholar; exclusion of anything not marine-specific; then a single consistent rubric—physics modeling, sensor simulation, rendering, system integration—applied across all entries in four comparison tables (one of which is a sensor-by-sensor matrix). Sorts the field into five generational paradigms: legacy (UWSim, UUV Simulator), physics-focused (Stonefish, DAVE), game-engine (HoloOcean, UNav-Sim, MARUS, FURo-Sim), GPU-native/Omniverse (OceanSim), and other (acoustics-first tools such as ACSim). The framing argument is that synthetic-data demand from DRL and visual SLAM, not vehicle dynamics, is what now drives simulator design. Open access, CC BY.

Song, Jingyu, Haoyu Ma, Onur Bagoren, Advait V. Sethuraman, Yiting Zhang, and Katherine A. Skinner (2025). “OceanSim: A GPU-Accelerated Underwater Robot Perception Simulation Framework”. In: *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. arXiv: [2503.01074](https://arxiv.org/abs/2503.01074). URL: <https://github.com/umfieldrobotics/OceanSim> (visited on 05/21/2026).

Notes. Brings underwater simulation onto NVIDIA Isaac Sim / Omniverse, with GPU ray tracing and a USD asset pipeline. Lead group: Field Robotics Group (FRoG), University of Michigan; PI Katherine A. Skinner. Apache-2.0; very active (v4.5 released Sept 2025, Isaac Sim 5.0 compatible). The strongest current candidate for vectorized ML training in marine perception. Underwater-only today, but the underlying engine extends naturally to surface, aerial, and terrestrial assets.

Zhang, Mabel M., Woen-Sug Choi, Jonathan Herman, Duane Davis, Carson Vogt, Michael McCarrin, Vaibhav Vijay, Dharini Dutia, William Lew, Steven Peters, and Brian Bingham (2022). “DAVE Aquatic Virtual Environment: Toward a General Underwater Robotics Simulator”. In: *Proc. IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*. Singapore. arXiv: [2209.02862](https://arxiv.org/abs/2209.02862). URL: <https://github.com/Field-Robotics-Lab/dave> (visited on 05/21/2026).

Notes. Extends UUV Simulator with standardized environments (bathymetry, currents), manipulation, and oceanographic data. Multi-institutional (NPS, WHOI, Open Robotics) and a port to modern Gazebo is in progress. Apache-2.0; documentation site at <https://field-robotics-lab.github.io/dave.doc/>. Included as the most ambitious extension of the UUV Sim lineage.